

pandas, on one card

A DataFrame is a spreadsheet you drive with code. Look at it, filter it, group it, join it, clean it -- in that order. Session 3, and the backbone of everything after.

Dr. Girish Kulkarni

girishmkulkarni.github.io/applied-maths-in-industry-site

Get the data in

Read a file

```
import pandas as pd
```

```
sales = pd.read_csv("shop_sales.csv")
sales = pd.read_excel("book.xlsx", sheet_name="Pune")
book = pd.read_excel("book.xlsx", sheet_name=None) # every sheet
```

```
na_values=["NA", "-"] treat these as blank
sep=";" not comma separated
thousands="," 1,200 -> 1200
parse_dates=["date"] read dates as dates
nrows=100 just peek at the top
```

Look before you compute

```
sales.head() # first 5 rows
sales.shape # (rows, columns)
list(sales.columns) # the column names
sales.info() # types + how many are filled
sales.describe() # min/mean/max of the numbers
sales.dtypes # what type is each column
```

→ Six lines, thirty seconds. They catch the problem before you build anything on top of it.

Build one by hand

```
orders = pd.DataFrame({
    "customer": ["Asha", "Ravi", "Asha"],
    "amount": [1200, 800, 450],
})
```

Two words worth keeping straight

Series one column. Has an index and values.
DataFrame many Series sharing one index.
index the row labels down the left

→ Most confusing pandas errors are really "I have a Series and I thought I had a DataFrame". Print type(x) to settle it.

Pick out what you want

Columns

```
sales["units"] # one column -> a Series
sales[["customer", "city"]] # several -> a DataFrame
sales["city"].unique() # the distinct values
sales["city"].value_counts() # how many of each
sales["city"].value_counts(normalize=True) * 100 # as %
```

! One pair of brackets gives a Series, two gives a DataFrame. That is the whole difference.

Rows: .loc by name, .iloc by number

```
sales.loc[0, "customer"] # by label
sales.loc[2, ["customer", "city"]]
sales.iloc[0] # by position
sales.iloc[0:3] # first three rows
```

! .loc[0:3] includes row 3. .iloc[0:3] stops before it. Labels are inclusive, positions are not.

Filter with a condition

```
bought_a_lot = sales["units"] > 5 # a True/False mask
big_buyers = sales[bought_a_lot] # keep those rows
```

```
sales[(sales["city"] == "Pune") & (sales["units"] > 5)]
sales[sales["city"].isin(["Mumbai", "Delhi"])]
sales[~sales["city"].isin(["Goa"])] # ~ means NOT
```

! Use & | ~, never and/or/not, and wrap every test in its own brackets. & binds tighter than >.

New columns, and sorting

```
sales["revenue"] = sales["units"] * sales["price"]
sales.sort_values("revenue", ascending=False)
sales.sort_values(["city", "revenue"])
sales.nlargest(5, "revenue") # top 5, no sorting needed
```

Numbers out of one column

```
.sum() .mean() total, average
.median() .std() middle, spread
.min() .max() extremes
.count() how many are filled
.nunique() how many distinct
.quantile(0.95) the 95th percentile
.describe() all of the above at once
```

Summarise and combine

groupby -- the pivot table idea

```
orders.groupby("customer")["amount"].sum()
orders.groupby("customer")["amount"].mean()
orders.groupby("customer")["amount"].count()
```

```
orders.groupby("city")["revenue"].agg(["count", "mean", "sum"])
orders.groupby(["city", "tier"])["amount"].sum()
```

→ Split into groups, apply a summary, combine the answers. Read it aloud: group by customer, take the amount column, add it up.

merge -- the VLOOKUP idea

```
pd.merge(orders, tiers, on="customer")
pd.merge(orders, tiers, on="customer", how="left")
how="inner" only rows in BOTH (the default)
how="left" keep all of the left table
how="outer" keep everything from both
```

```
indicator=True adds _merge: which side each row came from
validate="1:1" raise if the keys are not unique
```

! Rows vanishing after a merge is almost always an inner join, or keys that differ by stray spaces or case. Rows MULTIPLYING means duplicate keys. Check .shape before and after, every time.

Stack tables on top of each other

```
everything = pd.concat([book[name] for name in regions],
                       ignore_index=True)
```

→ concat stacks, merge joins side by side. ignore_index=True rennumbers the rows 0,1,2...

Reshape a table

BEYOND CLASS

```
grid = flights.pivot(index="month", columns="year",
                     values="passengers")
sales.pivot_table(index="city", columns="tier",
                  values="revenue", aggfunc="sum")
sales.melt(id_vars="city") # wide -> long
```

→ pivot just rearranges; pivot_table also summarises when several rows land in the same cell.

Clean a messy file

Diagnose first, then fix

```
raw.isna().sum() # gaps, per column
raw.duplicated().sum() # repeated rows
raw.dtypes # is price really text?
```

```
clean = raw.copy() # never edit the original
```

The cleaning kit, in order

```
clean = clean.drop_duplicates()
```

```
# sentinel -> a real blank
```

```
clean["units"] = clean["units"].replace(999, np.nan)
```

```
# text that should be a number
```

```
clean["price"] = clean["price"].astype(str).str.replace(
    ",", "", regex=False)
```

```
clean["price"] = pd.to_numeric(clean["price"], errors="coerce")
```

```
# fill the gaps
```

```
clean["units"] = clean["units"].fillna(clean["units"].mean())
```

```
# tidy the text so joins work
```

```
clean["city"] = clean["city"].str.strip().str.title()
```

```
# tidy the column NAMES too
```

```
clean.columns = (clean.columns.str.strip()
                 .str.lower().str.replace(" ", "_"))
```

→ errors="coerce" turns anything unparseable into NaN instead of crashing -- then isna() shows you exactly what was wrong.

Label rows with .loc

```
clean["order_size"] = "big"
```

```
clean.loc[clean["units"] < 5, "order_size"] = "small"
```

! NEVER write df[mask]["col"] = value. It edits a temporary copy and your change silently vanishes (SettingWithCopyWarning). One .loc, both sides.

Missing data

.isna()	.notna()	where are the gaps
.dropna()		drop rows with any gap
.dropna(subset=["price"])		only if THAT column is empty
.fillna(0)		fill with a value
.fillna(df["x"].median())		fill with the middle value

! NaN is not equal to anything, including itself. Test with .isna(), never with == None.

Write it back out

```
clean.to_csv("clean.csv", index=False)
```

```
clean.to_excel("clean.xlsx", index=False)
```

```
with pd.ExcelWriter("book.xlsx", engine="openpyxl") as writer:
    clean.to_excel(writer, sheet_name="clean_data", index=False)
```

! index=False, or you get a stray unnamed column every time the file is read back.

Rolling windows

BEYOND CLASS

```
prices["ma21"] = prices["close"].rolling(21).mean()
prices["vol"] = prices["close"].pct_change().rolling(21).std()
```

→ You meet these again in the finance sessions.

Tidying the frame itself

BEYOND CLASS

df.rename(columns={"a": "b"})	rename a column
df.drop(columns=["a"])	remove a column
df.set_index("id")	make a column the index
df.reset_index()	index back to a column
df.query("units > 5")	filter, written as text
pd.cut(df["x"], 3)	numbers -> bands
pd.get_dummies(df, columns=["city"])	text -> 0/1 columns

Text columns: the .str accessor

BEYOND CLASS

```
df["city"].str.strip().str.title()
df["city"].str.contains("pur", case=False)
df["code"].str.replace(",", "", regex=False)
df["name"].str.split(" ").str[0] # first word
df["city"].str.len()
```

→ Anything a Python string can do, .str does to a whole column at once.

Your own function, column-wide

BEYOND CLASS

```
df["tier"] = df["amount"].map(tier) # one column
df["z"] = df.apply(lambda r: r.a * r.b, axis=1) # whole row
```

! Reach for these last. A vectorised expression or .loc is faster and easier to read; apply is a loop wearing a disguise.

A chart, straight from the frame

BEYOND CLASS

```
sales.groupby("city")["revenue"].sum().plot(kind="bar")
sales.plot(x="units", y="revenue", kind="scatter")
```

→ Fine for a quick look. For anything you will show someone, use the seaborn recipes from Session 5.

Keep pandas current

! df.append() was removed in 2.0. Use pd.concat([df, other]).

! df.applymap() is now df.map(). Series.map() is unchanged.

! inplace=True buys you nothing: many methods copy under the hood anyway, and it breaks chaining. Write df = df.something(...).

! From pandas 3.0, copy-on-write means chained assignment simply stops working -- and it will not reliably tell you. Writing .loc properly today is the only safe habit.

Reading data anywhere

BEYOND CLASS

```
from pathlib import Path
```

```
path = Path("../data/shop_sales.csv")
```

```
if path.exists():
    sales = pd.read_csv(path) # your own machine
```

```
else:
    sales = pd.read_csv(COURSE_URL) # on Colab, from the web
```

→ The notebooks use this pattern so the same file runs on your laptop and on Colab without editing.

Counting two things at once

BEYOND CLASS

```
pd.crosstab(sales["city"], sales["tier"])
pd.crosstab(sales["city"], sales["tier"], normalize="index")
```

```
sales.groupby("city").agg(
    orders=("units", "count"),
    revenue=("revenue", "sum"),
    best=("revenue", "max"),
)
```

→ The named form gives you tidy column names instead of a stacked two-level header.

Chaining, once you are comfortable

BEYOND CLASS

```
report = (sales
         .query("units > 5")
         .groupby("city")["revenue"]
         .sum()
         .sort_values(ascending=False)
         .head(10))
```

→ Wrap the whole thing in brackets and put one step per line. It reads top to bottom as a sentence, and you can comment out any single step to debug it.

Mistakes everyone makes once

! A column of numbers showing dtype object means one stray piece of text is hiding in it. pd.to_numeric(..., errors="coerce") finds it.

! Most methods RETURN a new frame and change nothing. df.drop_duplicates() alone does nothing; write df = df.drop_duplicates().

! Arithmetic between two Series lines up on the INDEX, not on position. After filtering, reset_index(drop=True) if you are about to combine them.

! groupby silently DROPS rows whose key is blank. Pass dropna=False if those rows matter.

! One blank turns a whole integer column into floats (5 becomes 5.0). Expected, not a bug.

! head() showing what you expect is not a check. .shape and .isna().sum() are.

Worth knowing next

Dates

BEYOND CLASS

```
df["date"] = pd.to_datetime(df["date"])
df["date"].dt.year # .month .day .dayofweek
df["date"].dt.month_name()
df.set_index("date").resample("ME")["sales"].sum()
```